

Graph Algorithms

Compute Connectivity, Paths, and Structures Over Graphs

Minseok Jeon
DGIST

November 2, 2025

Table of Contents

1. Introduction
2. BFS/DFS Applications
3. Shortest Paths: Dijkstra/Bellman-Ford
4. Minimum Spanning Trees: Kruskal/Prim
5. Topological Sort and DAG DP
6. Strongly Connected Components
7. Flow Algorithms Overview
8. Summary

Introduction

What are Graph Algorithms?

Graph Algorithms: Methods to solve problems on graph structures

Key Problems:

- **Traversal:** Visit all vertices/edges
- **Connectivity:** Determine if vertices are connected
- **Shortest Paths:** Find minimum cost paths
- **Spanning Trees:** Connect all vertices with minimum cost
- **Flow:** Maximize throughput in networks

Real-World Applications:

- Social networks (connections, recommendations)
- Navigation systems (GPS, routing)
- Computer networks (packet routing, topology)
- Task scheduling (dependencies, ordering)
- Bioinformatics (protein interactions, gene networks)

Graph Basics Recap

Graph: $G = (V, E)$ where V = vertices, E = edges

Types:

- **Directed vs Undirected:** Edges have direction or not
- **Weighted vs Unweighted:** Edges have costs or not
- **Cyclic vs Acyclic:** Contains cycles or not (DAG)
- **Connected vs Disconnected:** All vertices reachable or not

Complexity Notation:

- V = number of vertices
- E = number of edges
- Dense graph: $E \approx V^2$
- Sparse graph: $E \approx V$

BFS/DFS Applications

Breadth-First Search (BFS)

Level-by-Level Exploration Using Queue

Algorithm:

1. Start from source vertex
2. Add to queue and mark visited
3. Dequeue, process, enqueue unvisited neighbors
4. Repeat until queue empty

Characteristics:

- **Time:** $O(V + E)$
- **Space:** $O(V)$ for queue and visited set
- **Data Structure:** Queue (FIFO)

Key Property:

- Finds **shortest path** in unweighted graphs
- Visits vertices in order of distance from source

BFS Implementation

```
1 from collections import deque
2
3 def bfs(graph, start):
4     """BFS traversal from start node"""
5     visited = set([start])
6     queue = deque([start])
7     result = []
8
9     while queue:
10         node = queue.popleft()
11         result.append(node)
12
13         for neighbor in graph[node]:
14             if neighbor not in visited:
15                 visited.add(neighbor)
16                 queue.append(neighbor)
17
18     return result
19
20 # Example
21 graph = {
22     0: [1, 2],
23     1: [0, 3, 4],
24     2: [0, 4],
25     3: [1],
26     4: [1, 2]
27 }
28 print(bfs(graph, 0)) # [0, 1, 2, 3, 4]
```

BFS Applications

1. Shortest Path in Unweighted Graph:

- Find minimum number of edges from source to target
- Track parent pointers to reconstruct path

2. Level-Order Traversal:

- Process nodes by distance from source
- Used in tree level-order traversal

3. Connected Components:

- Find all disconnected subgraphs
- Run BFS from each unvisited vertex

4. Bipartite Check:

- Determine if graph is 2-colorable
- Alternate colors during BFS

5. Web Crawling:

- Explore web pages level by level

Depth-First Search (DFS)

Explore as Deep as Possible Using Stack/Recursion

Algorithm:

1. Start from source vertex
2. Mark visited, explore first unvisited neighbor
3. Recursively DFS on neighbor
4. Backtrack when no unvisited neighbors

Characteristics:

- **Time:** $O(V + E)$
- **Space:** $O(V)$ for recursion stack (or explicit stack)
- **Data Structure:** Stack (LIFO) or recursion

Key Property:

- Explores entire branch before backtracking
- Can detect cycles (back edges)

DFS Implementation

```
1 def dfs_recursive(graph, node, visited=None):
2     """DFS traversal (recursive)"""
3     if visited is None:
4         visited = set()
5
6     visited.add(node)
7     result = [node]
8
9     for neighbor in graph[node]:
10         if neighbor not in visited:
11             result.extend(dfs_recursive(graph, neighbor, visited))
12
13     return result
14
15 def dfs_iterative(graph, start):
16     """DFS traversal (iterative)"""
17     visited = set()
18     stack = [start]
19     result = []
20
21     while stack:
22         node = stack.pop()
23         if node not in visited:
24             visited.add(node)
25             result.append(node)
26             stack.extend(reversed(graph[node])) # Maintain order
27
28     return result
```

DFS Applications

1. Cycle Detection:

- Find back edges (edge to ancestor)
- Detect if graph contains cycles

2. Path Finding:

- Find any path between two nodes
- Find all paths (with backtracking)

3. Topological Sorting:

- Order vertices in DAG
- Process finish times in reverse

4. Maze Solving:

- Find path through grid
- Backtrack on dead ends

5. Strongly Connected Components:

- Kosaraju's and Tarjan's algorithms

BFS vs DFS Comparison

Feature	BFS	DFS
Data Structure	Queue	Stack/Recursion
Path Found	Shortest	Any path
Memory (worst)	$O(V)$ (wide)	$O(h)$ (height)
Best For	Shortest path	Cycle detection
Tree Traversal	Level-order	Pre/In/Post-order
Completeness	Yes	Yes

When to Use BFS:

- Find shortest path in unweighted graph
- Process nodes by distance
- Graph is very deep (avoid stack overflow)

When to Use DFS:

- Detect cycles, find topological order
- Explore all paths, backtracking problems

Shortest Paths: Dijkstra/Bellman-Ford

Dijkstra's Algorithm

Single-Source Shortest Path (Non-Negative Weights)

Algorithm:

1. Initialize distances: source = 0, others = ∞
2. Use min-heap to get vertex with minimum distance
3. For each neighbor, relax edge if shorter path found
4. Mark vertex as visited
5. Repeat until all vertices processed

Characteristics:

- **Time:** $O((V + E) \log V)$ with min-heap
- **Space:** $O(V)$
- **Requirement:** Non-negative edge weights

Key Idea:

- Greedy approach: always pick closest unvisited vertex
- Correctness requires non-negative weights

Dijkstra Implementation

```
1 import heapq
2
3 def dijkstra(graph, start):
4     """Find shortest paths from start to all vertices"""
5     dist = {node: float('inf') for node in graph}
6     dist[start] = 0
7     pq = [(0, start)] # (distance, node)
8     visited = set()
9
10    while pq:
11        d, node = heapq.heappop(pq)
12        if node in visited:
13            continue
14        visited.add(node)
15
16        for neighbor, weight in graph[node]:
17            new_dist = d + weight
18            if new_dist < dist[neighbor]:
19                dist[neighbor] = new_dist
20                heapq.heappush(pq, (new_dist, neighbor))
21
22    return dist
23
24 # Example: graph[node] = [(neighbor, weight), ...]
25 graph = {
26     'A': [('B', 4), ('C', 2)],
27     'B': [('C', 1), ('D', 5)],
28     'C': [('D', 8), ('E', 10)],
29     'D': [('E', 2)],
30     'E': []
31 }
```

Bellman-Ford Algorithm

Single-Source Shortest Path (Handles Negative Weights)

Algorithm:

1. Initialize distances: source = 0, others = ∞
2. Relax all edges $V - 1$ times
3. Check for negative cycles (one more iteration)

Characteristics:

- **Time:** $O(V \times E)$
- **Space:** $O(V)$
- **Advantage:** Handles negative weights, detects negative cycles

Why $V - 1$ Iterations?

- Longest simple path has $V - 1$ edges
- Each iteration extends shortest path by one edge
- After $V - 1$ iterations, all shortest paths found

Bellman-Ford Implementation

```
1 def bellman_ford(edges, n, start):
2     """
3     Find shortest paths, detect negative cycles
4     edges: list of (u, v, weight)
5     """
6     dist = [float('inf')] * n
7     dist[start] = 0
8
9     # Relax edges V-1 times
10    for _ in range(n - 1):
11        for u, v, weight in edges:
12            if dist[u] != float('inf') and dist[u] + weight < dist[v]:
13                dist[v] = dist[u] + weight
14
15    # Check for negative cycles
16    for u, v, weight in edges:
17        if dist[u] != float('inf') and dist[u] + weight < dist[v]:
18            return None # Negative cycle detected
19
20    return dist
21
22 # Example
23 edges = [(0, 1, 4), (0, 2, 2), (1, 2, -3), (2, 3, 2), (3, 1, 1)]
24 n = 4
25 print(bellman_ford(edges, n, 0))
```

Dijkstra vs Bellman-Ford

Feature	Dijkstra	Bellman-Ford
Time Complexity	$O(E \log V)$	$O(V \times E)$
Negative Weights	No	Yes
Negative Cycles	Cannot detect	Detects
Implementation	Min-heap	Nested loops
Best For	Fast, non-negative	Negative weights

Floyd-Warshall (All-Pairs Shortest Paths):

- **Time:** $O(V^3)$
- Finds shortest paths between all pairs
- Handles negative weights
- Uses dynamic programming

Choosing Algorithm:

- Non-negative weights → Dijkstra (faster)
- Negative weights → Bellman-Ford

Minimum Spanning Trees: Kruskal/Prim

Minimum Spanning Tree (MST)

Tree Connecting All Vertices with Minimum Total Weight

Properties:

- Connects all V vertices
- Has exactly $V - 1$ edges
- No cycles (it's a tree)
- Minimum total edge weight

Applications:

- Network design (minimize cable length)
- Approximation algorithms (TSP)
- Clustering algorithms
- Image segmentation

Two Main Algorithms:

- **Kruskal's:** Sort edges, add if no cycle (edge-based)
- **Prim's:** Grow tree from vertex (vertex-based)

Kruskal's Algorithm

Sort Edges, Add If Doesn't Create Cycle

Algorithm:

1. Sort all edges by weight (ascending)
2. Initialize Union-Find structure
3. For each edge (u, v) :
 - If u and v in different components, add edge
 - Union the components
4. Stop when $V - 1$ edges added

Characteristics:

- **Time:** $O(E \log E)$ (dominated by sorting)
- **Space:** $O(V)$ for Union-Find
- **Best for:** Sparse graphs

Key Data Structure:

- Union-Find (Disjoint Set Union) for cycle detection

Kruskal Implementation

```
1 class UnionFind:
2     def __init__(self, n):
3         self.parent = list(range(n))
4         self.rank = [0] * n
5
6     def find(self, x):
7         if self.parent[x] != x:
8             self.parent[x] = self.find(self.parent[x]) # Path compression
9         return self.parent[x]
10
11     def union(self, x, y):
12         px, py = self.find(x), self.find(y)
13         if px == py:
14             return False # Already in same set
15         if self.rank[px] < self.rank[py]:
16             self.parent[px] = py
17         elif self.rank[px] > self.rank[py]:
18             self.parent[py] = px
19         else:
20             self.parent[py] = px
21             self.rank[px] += 1
22         return True
23
24 def kruskal(n, edges):
25     """edges: list of (weight, u, v)"""
26     edges.sort() # Sort by weight
27     uf = UnionFind(n)
28     mst, total = [], 0
29
30     for weight, u, v in edges:
```

Prim's Algorithm

Grow Tree from Starting Vertex

Algorithm:

1. Start with any vertex
2. Add to MST
3. Repeat until all vertices added:
 - Find minimum weight edge from MST to non-MST vertex
 - Add that edge and vertex to MST

Characteristics:

- **Time:** $O(E \log V)$ with min-heap
- **Space:** $O(V)$
- **Best for:** Dense graphs

Key Data Structure:

- Min-heap to efficiently find minimum edge

Prim Implementation

```
1 import heapq
2
3 def prim(graph, start):
4     """
5     graph: {node: [(neighbor, weight), ...]}
6     """
7     mst, total = [], 0
8     visited = {start}
9     edges = [(weight, start, neighbor)
10              for neighbor, weight in graph[start]]
11     heapq.heapify(edges)
12
13     while edges:
14         weight, u, v = heapq.heappop(edges)
15         if v in visited:
16             continue
17
18         visited.add(v)
19         mst.append((u, v, weight))
20         total += weight
21
22         for neighbor, w in graph[v]:
23             if neighbor not in visited:
24                 heapq.heappush(edges, (w, v, neighbor))
25
26     return total, mst
27
28 # Example
29 graph = {
30     'A': [( 'B', 4), ( 'C', 2)],
31     'B': [( 'A', 4), ( 'C', 1), ( 'D', 5)],
```

Kruskal vs Prim

Feature	Kruskal	Prim
Time Complexity	$O(E \log E)$	$O(E \log V)$
Approach	Edge-based	Vertex-based
Data Structure	Union-Find	Min-heap
Best For	Sparse graphs	Dense graphs
Starting Point	N/A (all edges)	Any vertex
Works on Disconnected	Partial MST	No

Time Complexity Notes:

- Kruskal: $O(E \log E) = O(E \log V)$ since $E \leq V^2$
- Prim with Fibonacci heap: $O(E + V \log V)$ (theoretical)

Practical Choice:

- Sparse graph ($E \approx V$): Kruskal slightly better
- Dense graph ($E \approx V^2$): Prim slightly better
- Both give same MST weight (may differ in edges)

Topological Sort and DAG DP

Topological Sort

Linear Ordering of Vertices in DAG

Definition:

- For every directed edge (u, v) , u comes before v in ordering
- Only exists for Directed Acyclic Graphs (DAGs)
- Can be multiple valid orderings

Applications:

- **Course scheduling:** Prerequisite dependencies
- **Build systems:** Compile dependencies (Makefile)
- **Task scheduling:** Task dependencies
- **Formula evaluation:** Dependency graphs

Two Main Algorithms:

- **Kahn's Algorithm:** BFS-based, uses in-degrees
- **DFS-based:** Process vertices by finish time

Topological Sort: Kahn's Algorithm

```
1 from collections import deque
2
3 def topological_sort(graph, n):
4     """
5     Kahn's algorithm (BFS-based)
6     graph: adjacency list
7     Returns: topological order or None if cycle exists
8     """
9     # Calculate in-degrees
10    in_degree = [0] * n
11    for node in range(n):
12        for neighbor in graph[node]:
13            in_degree[neighbor] += 1
14
15    # Start with vertices having in-degree 0
16    queue = deque([i for i in range(n) if in_degree[i] == 0])
17    result = []
18
19    while queue:
20        node = queue.popleft()
21        result.append(node)
22
23        for neighbor in graph[node]:
24            in_degree[neighbor] -= 1
25            if in_degree[neighbor] == 0:
26                queue.append(neighbor)
27
28    # If not all vertices processed, graph has cycle
29    return result if len(result) == n else None
```

DAG Dynamic Programming

DP on Directed Acyclic Graphs

Key Idea:

- Process vertices in topological order
- Each vertex computed after all dependencies
- No cycles $\rightarrow$ no circular dependencies

Common Problems:

- **Longest/Shortest Path in DAG:** $O(V + E)$
- **Count paths:** Number of paths from source to sink
- **Critical Path Method:** Project scheduling

Template:

1. Compute topological order
2. Initialize DP array
3. Process vertices in topological order
4. Update DP based on edges

DAG DP: Longest Path

```
1 def longest_path_dag(graph, n):
2     """Find longest path in DAG"""
3     topo_order = topological_sort(graph, n)
4     if topo_order is None:
5         return None # Cycle exists
6
7     dp = [0] * n
8
9     for node in topo_order:
10         for neighbor in graph[node]:
11             dp[neighbor] = max(dp[neighbor], dp[node] + 1)
12
13     return max(dp)
14
15 # Example: Course scheduling with prerequisites
16 # Find longest chain of courses
17 graph = {
```

Strongly Connected Components

Strongly Connected Components (SCC)

Maximal Subgraphs Where Every Vertex Reaches Every Other

Definition:

- In directed graph, SCC is maximal set of vertices
- For any u, v in SCC, there exists path $u \rightarrow v$ and $v \rightarrow u$
- Condensation graph (SCC graph) is always a DAG

Applications:

- **2-SAT**: Satisfiability of Boolean formulas
- **Reachability queries**: Which vertices can reach which
- **Deadlock detection**: Circular dependencies
- **Web page ranking**: Identify tightly connected clusters

Algorithms:

- **Kosaraju's**: Two-pass DFS (simpler)
- **Tarjan's**: Single-pass DFS (more efficient)

Kosaraju's Algorithm

Two-Pass DFS Approach

```
1 def kosaraju_scc(graph, n):
2     """Find strongly connected components"""
3     # Step 1: DFS to get finish order
4     visited = [False] * n
5     finish_order = []
6
7     def dfs1(node):
8         visited[node] = True
9         for neighbor in graph[node]:
10             if not visited[neighbor]:
11                 dfs1(neighbor)
12         finish_order.append(node)
13
14     for i in range(n):
15         if not visited[i]:
16             dfs1(i)
17
18     # Step 2: Transpose graph
19     transpose = [[] for _ in range(n)]
20     for u in range(n):
21         for v in graph[u]:
22             transpose[v].append(u)
23
24     # Step 3: DFS on transpose in reverse finish order
25     visited = [False] * n
26     components = []
```

Kosaraju's Algorithm: How It Works

Three Steps:

Step 1: First DFS

- Run DFS on original graph
- Record finish times (when vertex fully explored)
- Vertices in same SCC finish close together

Step 2: Transpose Graph

- Reverse all edge directions
- If $u \rightarrow v$ in G , then $v \rightarrow u$ in G^T
- SCCs remain the same

Step 3: Second DFS

- Process vertices in reverse finish order
- Each DFS tree in G^T is one SCC

Complexity:

Flow Algorithms Overview

Maximum Flow Problem

Find Maximum Flow from Source to Sink

Definition:

- Given: Directed graph with edge capacities
- Find: Maximum amount of flow from source s to sink t
- Constraints: Flow $\leq$ capacity, flow conservation

Key Concepts:

- **Capacity:** Maximum flow on edge
- **Residual graph:** Remaining capacity after flow
- **Augmenting path:** Path with available capacity
- **Cut:** Partition of vertices into two sets

Max-Flow Min-Cut Theorem:

- Maximum flow value = Minimum cut capacity
- Fundamental result in network flow theory

Ford-Fulkerson Method

Find Augmenting Paths Until No More Exist

Algorithm:

1. Initialize flow to 0
2. While augmenting path exists:
 - Find augmenting path (any path with capacity)
 - Compute bottleneck capacity
 - Add flow along path
 - Update residual graph

Edmonds-Karp Algorithm:

- Ford-Fulkerson with BFS for finding paths
- **Time:** $O(V \times E^2)$ (guaranteed polynomial)
- Always finds shortest augmenting path

Characteristics:

- **Time:** $O(V \times E^2)$ for Edmonds-Karp

Edmonds-Karp Implementation

```
1 from collections import deque
2
3 def max_flow(capacity, source, sink):
4     """Edmonds-Karp algorithm for maximum flow"""
5     n = len(capacity)
6     flow = [[0] * n for _ in range(n)]
7
8     def bfs():
9         """Find augmenting path using BFS"""
10        parent = [-1] * n
11        visited = [False] * n
12        visited[source] = True
13        queue = deque([(source, float('inf'))])
14
15        while queue:
16            u, min_cap = queue.popleft()
17
18            for v in range(n):
19                if not visited[v] and capacity[u][v] - flow[u][v] > 0:
20                    visited[v] = True
21                    parent[v] = u
22                    new_cap = min(min_cap, capacity[u][v] - flow[u][v])
23
24                    if v == sink:
25                        return parent, new_cap
26                    queue.append((v, new_cap))
27
28        return None, 0
```

Flow Applications

1. Maximum Bipartite Matching:

- Model as flow problem
- Add source to left vertices, sink from right vertices
- Maximum flow = maximum matching

2. Minimum Cut:

- Find minimum capacity cut separating s and t
- Max flow = min cut (by theorem)
- Applications: Network reliability, image segmentation

3. Network Routing:

- Optimize data flow through network
- Consider bandwidth constraints

4. Assignment Problems:

- Match workers to tasks optimally
- Constraint satisfaction

Summary

Key Takeaways

Traversal Algorithms:

- **BFS**: Shortest path, level-order, $O(V + E)$
- **DFS**: Cycle detection, topological sort, $O(V + E)$

Shortest Path Algorithms:

- **Dijkstra**: Non-negative weights, $O(E \log V)$
- **Bellman-Ford**: Negative weights, detects cycles, $O(VE)$

Minimum Spanning Tree:

- **Kruskal**: Sort edges, Union-Find, $O(E \log E)$
- **Prim**: Grow tree, min-heap, $O(E \log V)$

Advanced Topics:

- **Topological Sort**: Order DAG vertices, $O(V + E)$
- **SCC**: Kosaraju's/Tarjan's, $O(V + E)$
- **Max Flow**: Edmonds-Karp, $O(VE^2)$

Complexity Summary

Algorithm	Time	Space
BFS	$O(V + E)$	$O(V)$
DFS	$O(V + E)$	$O(V)$
Dijkstra	$O((V + E) \log V)$	$O(V)$
Bellman-Ford	$O(VE)$	$O(V)$
Kruskal	$O(E \log E)$	$O(V)$
Prim	$O(E \log V)$	$O(V)$
Topological Sort	$O(V + E)$	$O(V)$
Kosaraju SCC	$O(V + E)$	$O(V)$
Edmonds-Karp	$O(VE^2)$	$O(V^2)$

Practice Problems

BFS/DFS:

- Number of islands (LeetCode 200)
- Word ladder (LeetCode 127)
- Course schedule (LeetCode 207, 210)

Shortest Paths:

- Network delay time (LeetCode 743)
- Cheapest flights (LeetCode 787)

MST:

- Min cost to connect all points (LeetCode 1584)

Advanced:

- Critical connections (LeetCode 1192)
- Alien dictionary (LeetCode 269)

Resources

Books:

- "Introduction to Algorithms" (CLRS) - Chapters 22-26
- "Algorithm Design" (Kleinberg & Tardos)

Online:

- VisuAlgo - Graph algorithm visualizations
- LeetCode - Graph problems
- Codeforces - Graph theory tutorials

Advanced Topics:

- A* search algorithm
- Network simplex for min-cost flow
- Hopcroft-Karp for bipartite matching
- Tarjan's algorithm for SCC